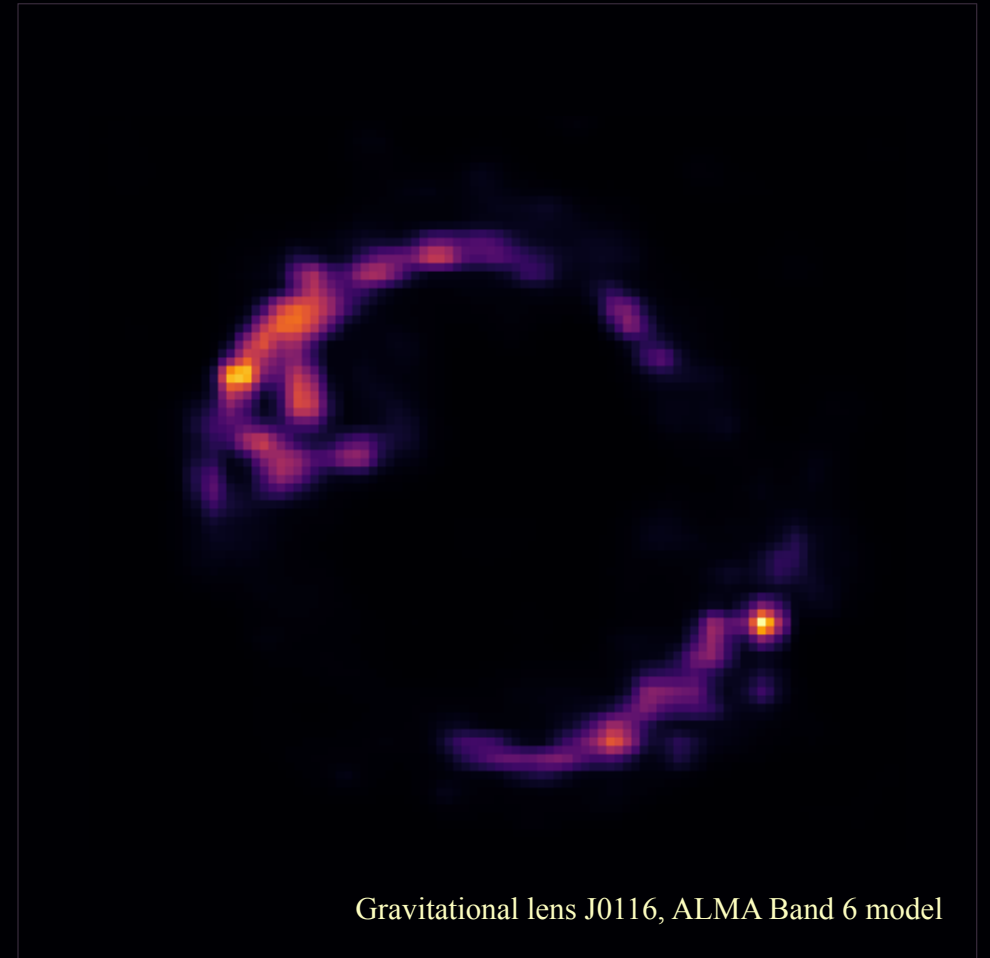


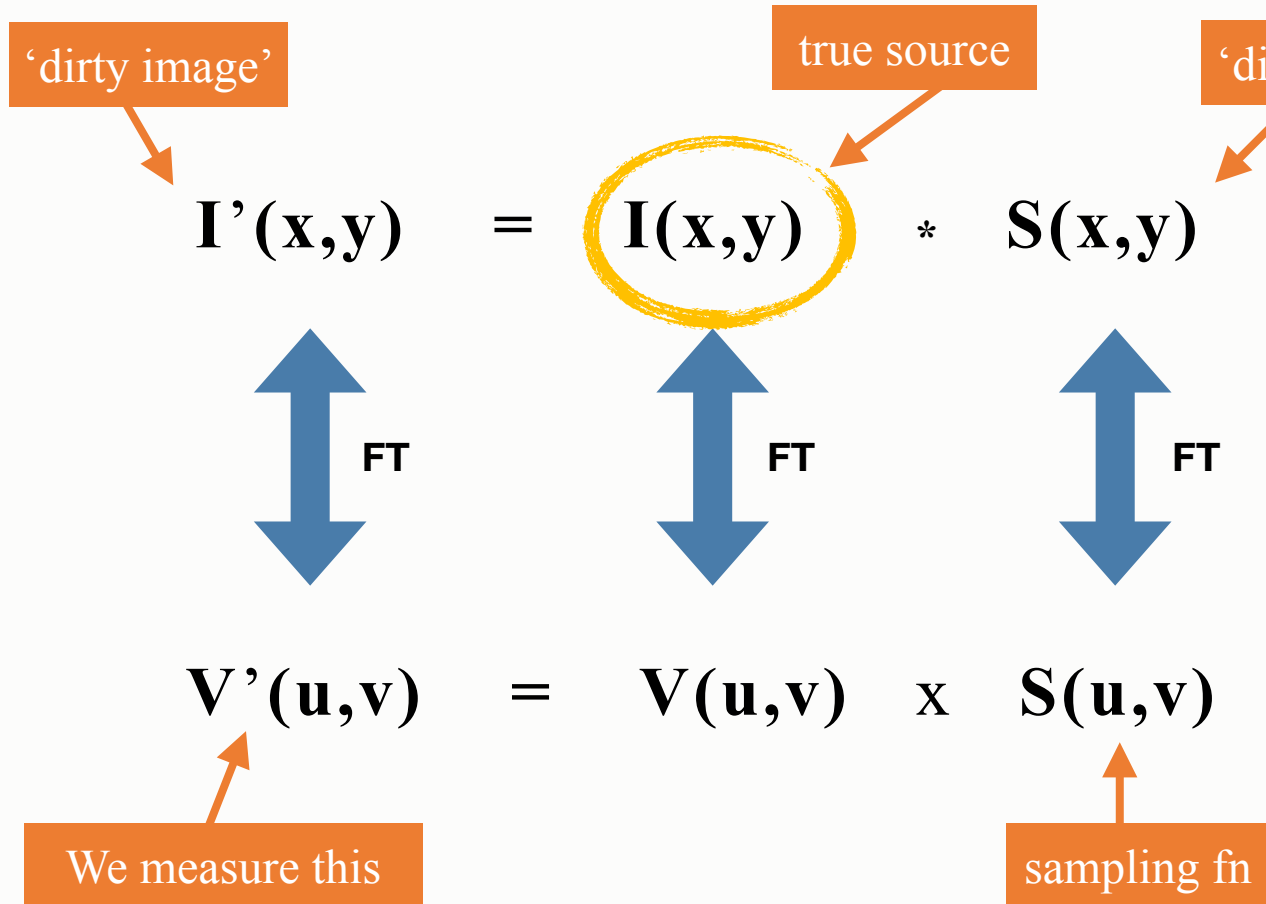
RADIO INTERFEROMETRIC IMAGING

pyuvimage

Freeform image reconstruction by forward
modelling of visibilities



The problem of image reconstruction



An interferometer samples **Fourier components** of the sky. Transform them back and you get the **dirty image**: the sky convolved with a messy beam.

The most common fix has been **CLEAN**: subtract point sources iteratively until the residual looks like noise.

It works and it's fast. But no likelihood, no error bars, introduces deconvolution artefacts, often not reproducible, and "clean deeply enough" is a judgement call.

Fit the sky to the visibilities directly

1 · SKY MODEL

A freeform image on a grid

No assumed shape. Every pixel is a free parameter.



2 · PREDICT

Fourier transform it

Predicted visibilities are a *linear* function of the pixels



3 · SOLVE

One linear inversion

Compare with the data and solve for the sky

THE PART THAT MATTERS

Unregularised, that just fits the noise. So the sky gets a **Gaussian-process prior** — neighbouring pixels are correlated over about a beam — and its strength and scale are **chosen automatically** from the data. Nothing to tune by hand.

The likelihood is a linear model

DATA MODEL

$$d = \mathbf{A} s + n$$

d — the data: real and imaginary parts of the N measured visibilities, length $2N$. s — the sky: one brightness per mesh pixel, N_{mesh} of them. $\mathbf{A}$ — the instrument: the Fourier transform from mesh pixels to the sampled (u, v) points, primary beam included. n — Gaussian noise, one σ_i per visibility.

LOG-LIKELIHOOD

$$\ln L(s) = -\frac{1}{2} \chi^2(s) + \text{const}$$

$$\chi^2(s) = (d - \mathbf{A}s)^T \mathbf{W} (d - \mathbf{A}s)$$

$\mathbf{W}$ — the noise weights, $\text{diag}(1/\sigma_i^2)$. $d - \mathbf{A}s$ — the residual visibilities. T — transpose. Everything is evaluated in the uv -plane, against the numbers the telescope measured — never against a dirty image.

MAXIMUM LIKELIHOOD IS A LINEAR SOLVE

$$\mathbf{F} s = D, \quad \mathbf{F} = \mathbf{A}^T \mathbf{W} \mathbf{A}, \quad D = \mathbf{A}^T \mathbf{W} d$$

‘dirty beam’

‘dirty image’

$\mathbf{F}$ — the curvature (Fisher) matrix, $N_{\text{mesh}} \times N_{\text{mesh}}$: how sharply χ^2 rises in each direction. D — the data vector: the weighted dirty image sampled on the mesh, up to a scale.

BUT WE CAN'T STOP HERE...

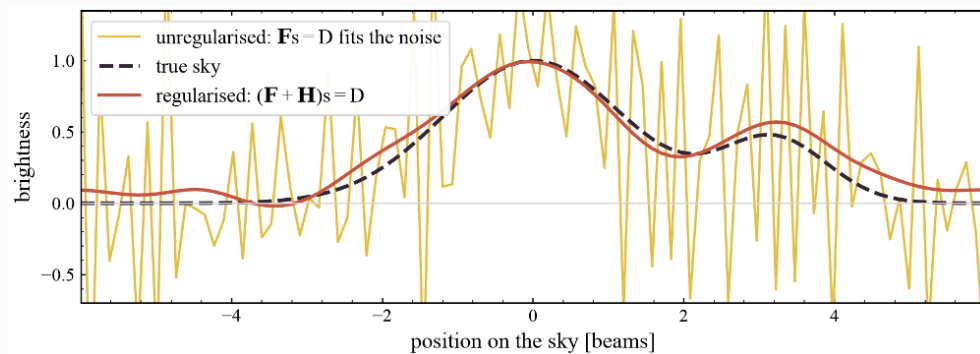
The uv -plane is sampled, not covered: whole spatial frequencies are unconstrained, so $\mathbf{F}$ is near-singular. The unregularised solution fits the noise perfectly and the sky not at all.

The regularised inversion

PRIOR: A GAUSSIAN PROCESS ON THE SKY

$$p(s) \propto \exp(-1/2 \lambda s^T \mathbf{C}^{-1} s), \quad \mathbf{H} \equiv \lambda \mathbf{C}^{-1}$$

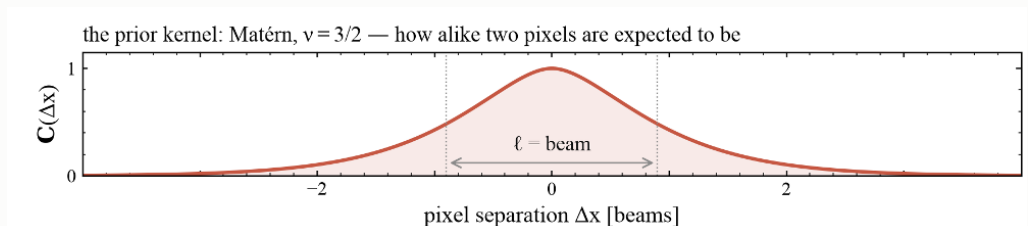
s — the sky on the mesh, as before. $\mathbf{C}$ — the prior covariance between mesh pixels: a Matérn kernel of smoothness $\nu = 3/2$ and correlation length $\ell = \sqrt{(b_{\text{maj}} \cdot b_{\text{min}})}$, the beam's geometric-mean FWHM — structure finer than the beam is not constrained by the data, so the prior supplies it. λ — the prior strength (regularisation coefficient), the one hyperparameter. $\mathbf{H}$ — the prior precision matrix. The mean is zero, so nothing is imposed on the flux.



POSTERIOR — STILL ONE LINEAR SOLVE

$$(\mathbf{F} + \mathbf{H}) s = \mathbf{D}, \quad \Sigma = (\mathbf{F} + \mathbf{H})^{-1}$$

$\mathbf{F}$, $\mathbf{D}$ — the curvature matrix and data vector from the likelihood. s here — the maximum-a-posteriori (MAP) sky, found by one Cholesky solve; a non-negative variant is optional. Σ — the posterior covariance of s , closed-form because likelihood and prior are both Gaussian. Mapped to the image grid by the interpolation matrix $\mathbf{M}$, $\sqrt{\text{diag}(\mathbf{M}\Sigma\mathbf{M}^T)}$ is the per-pixel statistical 1σ used later.



Choosing the strength λ — automatically

λ is bisected over 10^{-6} – 10^6 until one of three criteria is met; **--criterion auto** picks between the first two by how well the data constrain the mesh.

DISCREPANCY

$$\chi^2(\lambda) = N$$

The strongest smoothing that still fits the data to the noise (Morozov). N — the number of visibilities, so $\chi^2/N = 1$ means residuals of noise size on average. Default below 10 data points per mesh pixel. A χ^2 floor that cannot reach N is aimed at from $2\sqrt{(2/N)}$ above, the standard error of χ^2/N .

STRUCTURE

$$\text{rms}(r) / \sqrt{\chi^2/N} = 1$$

r — the residual *map*, the dirty image of $d - \mathbf{A}s$ in units of the image noise, compared with the rms it would have were the residual visibilities white. ~ 1 : noise. > 1 : structure the fit discarded. < 1 : the model absorbed noise. Default on well-constrained fits: on real data χ^2 is nearly flat in λ and the map is the sharper test.

EVIDENCE

$$\ln Z = -\frac{1}{2} [\chi^2 + s^T \mathbf{H} s + \ln |\mathbf{F} + \mathbf{H}| - \ln |\mathbf{H}| + \sum_i \ln 2\pi\sigma_i^2]$$

Maximise Z , the probability of the data with the sky marginalised out. s — the MAP sky at this λ ; $s^T \mathbf{H} s$ — its prior penalty; $|\cdot|$ — determinant, the two log-determinants the Occam factor against a prior stronger than needed; σ_i — noise per visibility. PyAutoLens' choice; best when visibilities outnumber pixels.

Scaling up: the $\tilde{W}$ kernel

F IS TRANSLATION-INVARIANT

$$F_{ij} = \sum_k w_k \cos 2\pi(u_k \Delta x + v_k \Delta y) = \tilde{W}(\Delta x, \Delta y)$$

F_{ij} — the curvature between mesh pixels i and j . $w_k = 1/\sigma_k^2$ — the weight of visibility k ; (u_k, v_k) — its baseline in wavelengths. $(\Delta x, \Delta y)$ — the sky separation of the two pixels. $\tilde{W}$ — that sum: a function of separation only, tabulated once on $(2N_y, 2N_x)$ points for an $N_y \times N_x$ image.

So one streaming pass accumulates $\tilde{W}$, F is read off it, and F_s is an FFT convolution — gridding, as in tclean. $\tilde{W}$ depends only on uv coverage, noise and geometry, so it is cached on disk.

WHAT IT COSTS — RUBY CO(7–6), 148K VISIBILITIES

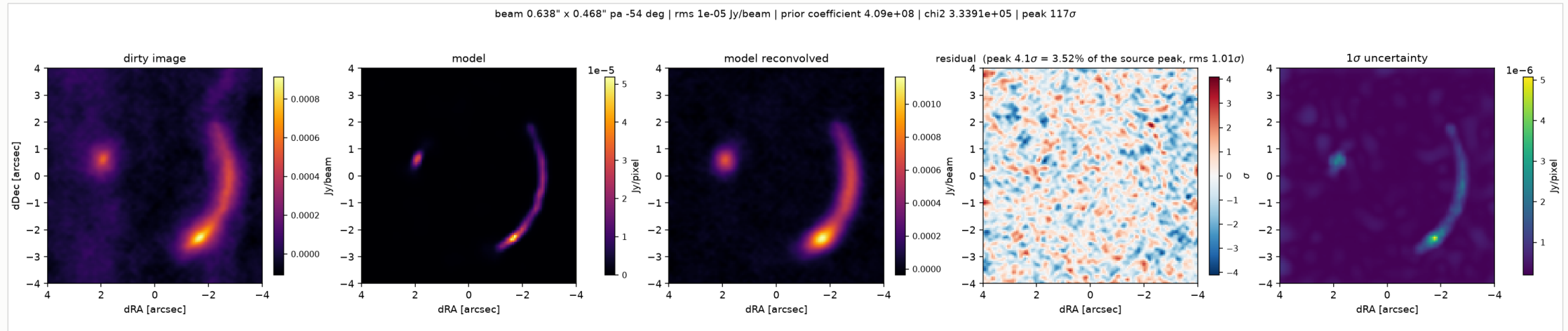
	dense	sparse ($\tilde{W}$)
largest allocation	21.6 GB	0.10 MB
scales with nvis	yes	no
full fit, 26×26 mesh	tens of minutes	~30 s
limited by	$N_{vis} \times N_{mesh}$	--mesh only

Streamed from a file, the data are never held at all: a 200-million-sample MFS cube that needs 33 GB in memory needs about 1 GB. The cost moves to time — one pass per sample.

STATUS

Chosen automatically above 5000 visibilities. Agrees with the dense path to 3×10^{-8} of the peak on the built-in mock. Needs JAX.

What you get out from the fit



1.016

χ^2/N — the model fits 164,262 visibilities to the noise

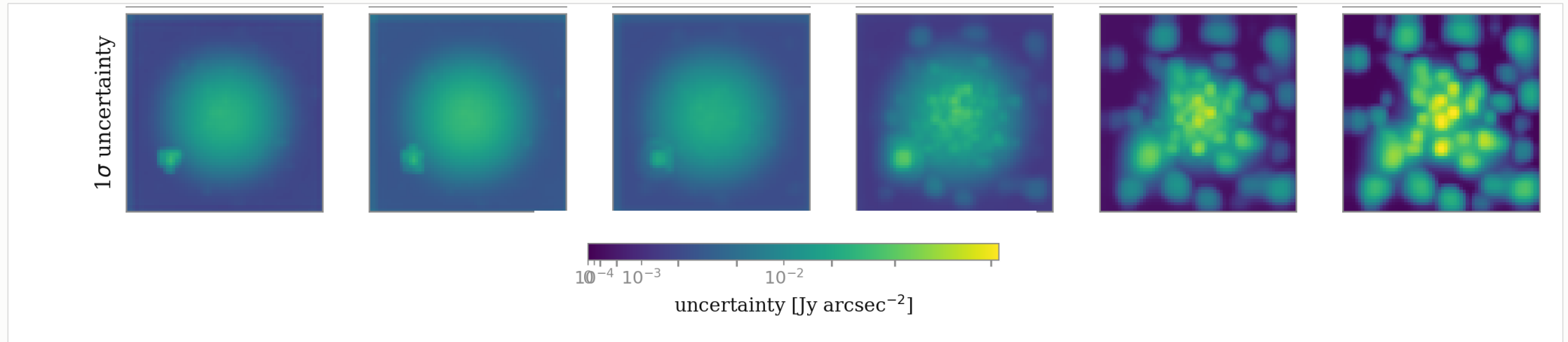
3.5%

the largest residual (4.1 σ) as a fraction of a 117 σ peak

1.01 σ

residual-map rms, with no trace of the source

Realistic(ish) uncertainties: a uncertainty 1σ map



STATISTICAL

How well the data pin this pixel down, given the prior. Closed-form posterior width, verified against Monte Carlo at 0.996. Dominates in smooth emission.

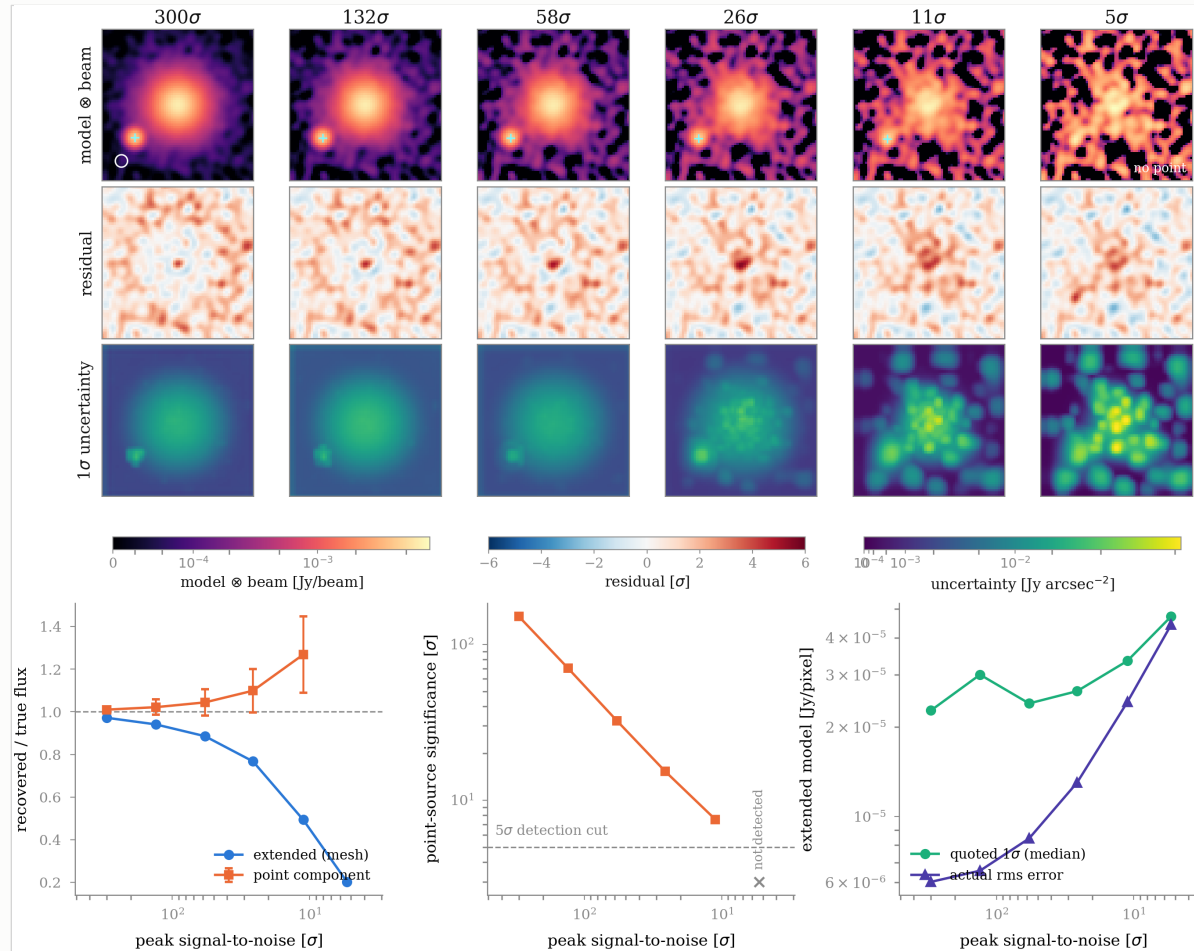
PRIOR SYSTEMATIC

How far the pixel moves over the range of smoothing the data cannot tell apart. Dominates on compact features — exactly where a statistical error bar alone would mislead.

TOTAL ERRORS

uncertainty.fits and snr.fits. Aperture errors use the full covariance: adding pixels in quadrature overstated a compact aperture by $\sim 1.4\times$ on the mocks.

Statistical tests from 300σ down to 5σ



Top rows: one source, six noise levels, colour scales shared along each row. The residual stays white; the uncertainty grows where it should.

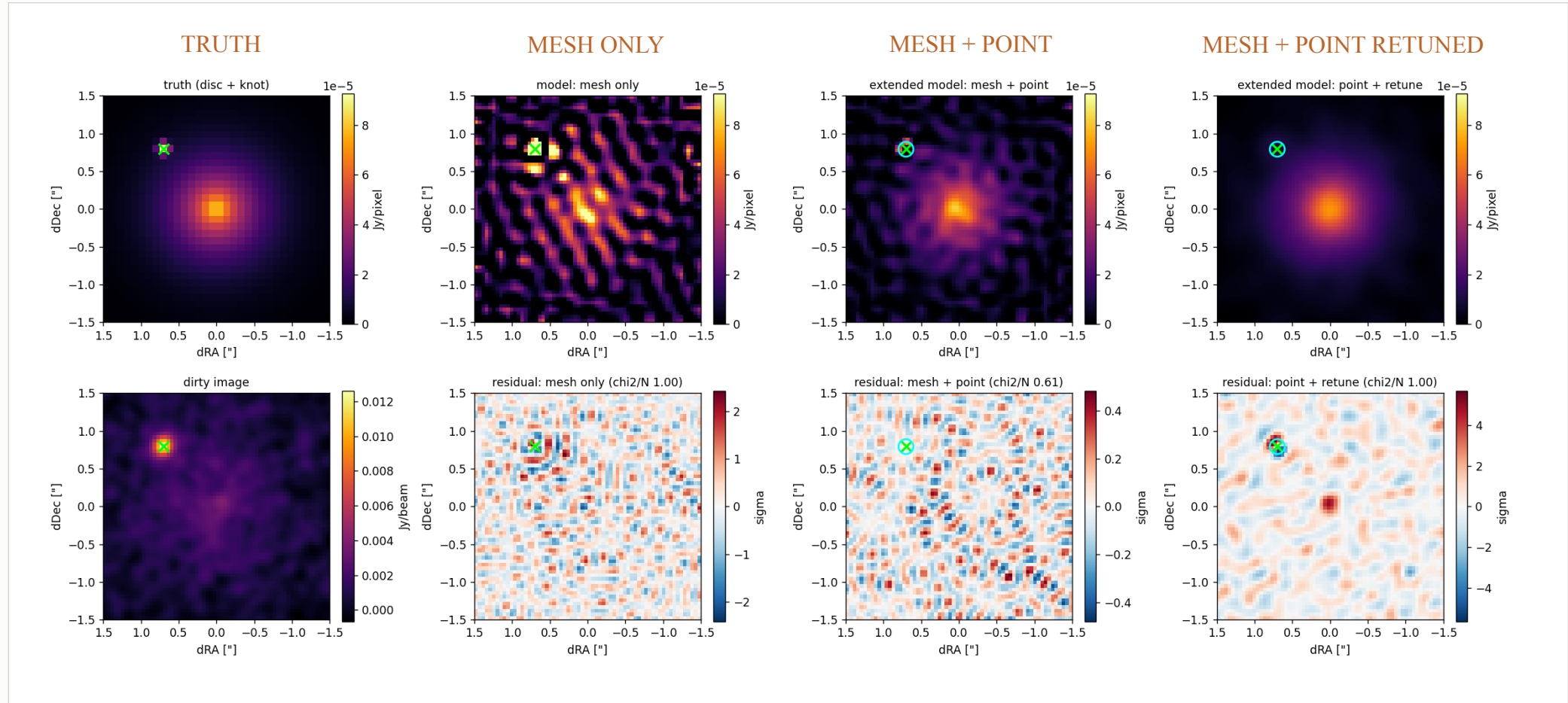
Bottom left: recovered flux against truth — the point component holds to 11σ , the extended emission drops out first

Bottom right: the quoted 1σ tracks the actual rms error where the fit is well constrained.

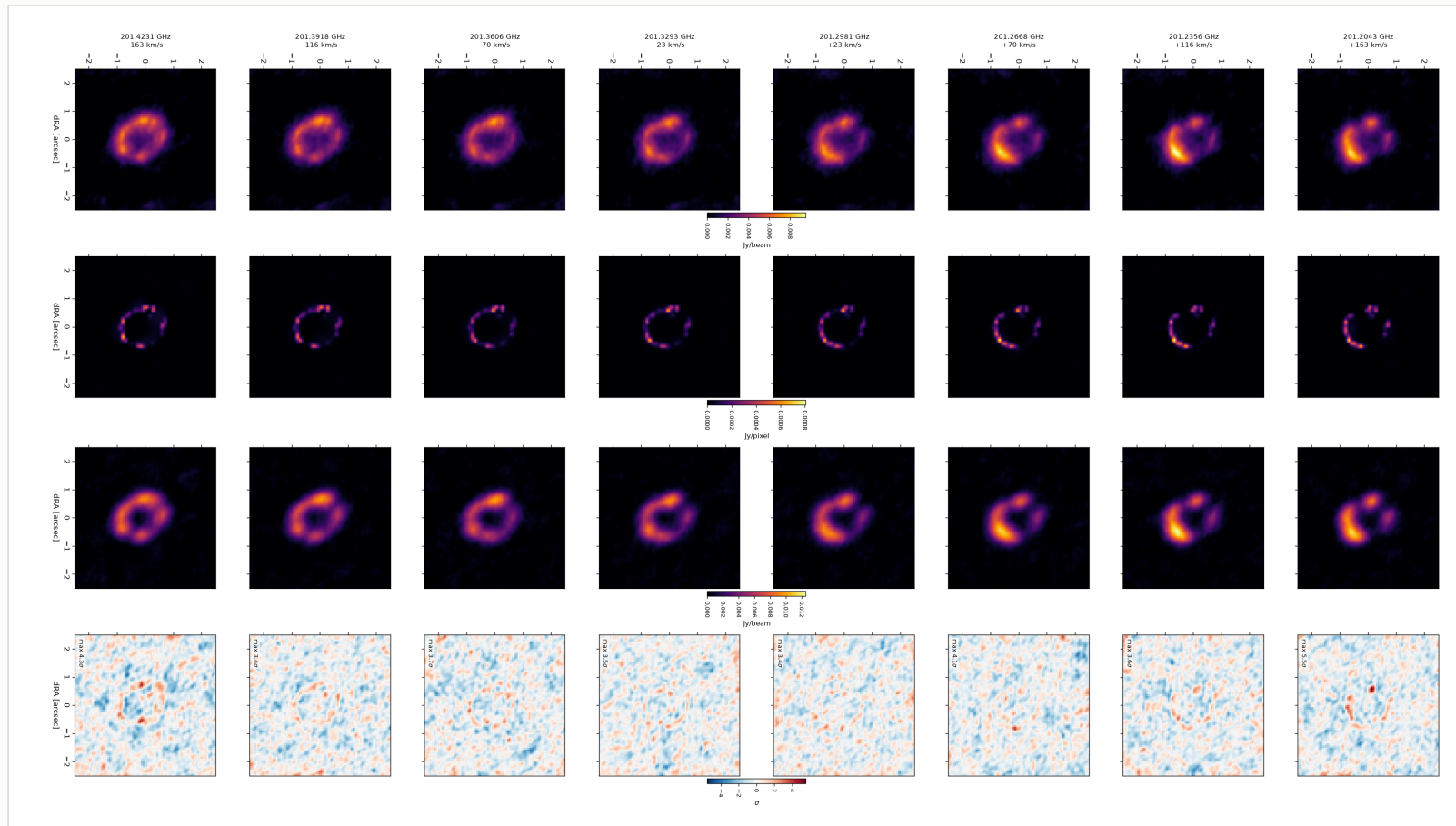
POINT SOURCES

A pixel grid cannot represent a true point. --point-sources adds analytic deltas solved in the same linear system. Opt-in, conservative, zero false positives across the test suite.

Including analytic point sources



Cube imaging



Example: ALMA band 6 data of CO from Ruby gravitational lens

CUBE IMAGING

--mode cube models each channel (**--mode mfs** is default)
Choose from **--cube-prior mfs** (default) and **--cube-prior channel** (latter is more expensive)

```
pyuvimage fit Ruby_CO7-6_48kms.npz --fov 5 --image-centre="-2,-2" --cube-prior mfs --mode cube
```

Try it on your data

--fov is the only required input. Everything else has a default that is meant to be left alone.

Runs on a laptop. Pure NumPy is fully supported; JAX is optional but *strongly recommended* above 5000 visibilities.

Current limitations: one field. No weighting schemes, mosaic, w-term or full pol.

GitHub link



FROM A MEASUREMENT SET

```
pip install -e "[ms]"  
pyuvimage import obs.ms mydata/  
pyuvimage fit mydata/ --fov 3.0
```

FROM PYTHON

```
import pyuvimage  
result = pyuvimage.run("mydata/", fov=3.0)
```

NO DATA AT ALL

```
pyuvimage demo
```